CSE 355 EC Project: Final Report

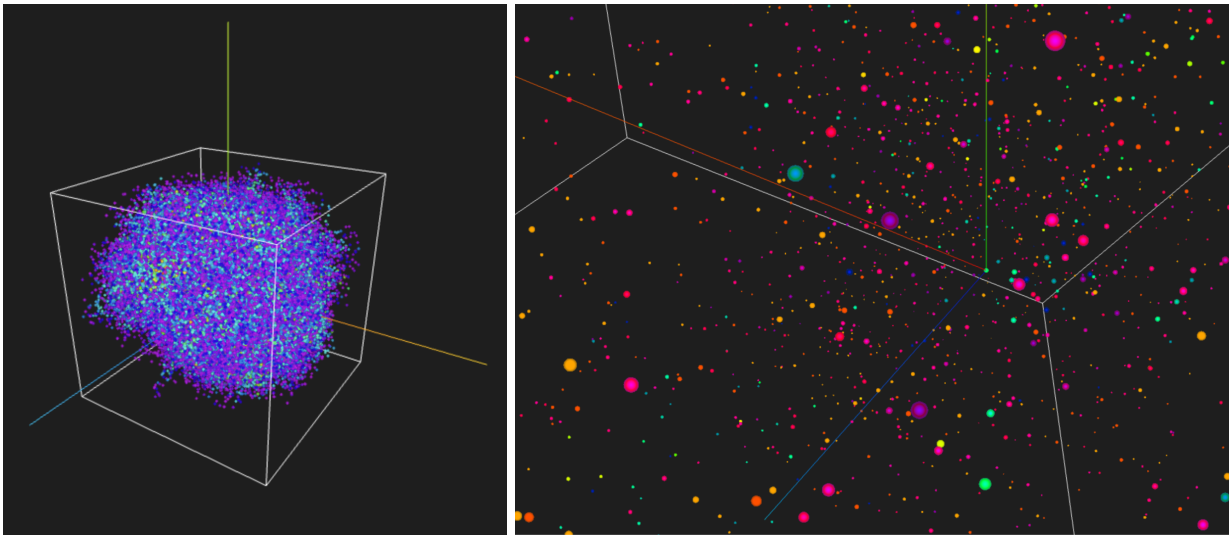
Arizona State University
School of Computing and Augmented Intelligence

Supervising Faculty:
Name: Hani Ben Amor
Email: hbenamor@asu.edu

Student:
Name: Sun-Yen Tan
Email: stan28@asu.edu
ASU ID: 1221757227

Course Information:
Course: CSE 355 Introduction to Theoretical Computer Science
Semester: Spring 2025

3D Cellular Automaton: Starlight



Purpose: This project aims to explore the realm of Cellular Automatons, where most of them are experimented in a 2D plane (x, y axis). Using three.js, I will be exploring 3 dimensional cellular automatons, where I have an additional z-axis. This project takes inspiration from star formation and the gravitational pull in the solar system. The automaton follows stochastic growth rules and is modeled using principles from the theory of computation, including context-free grammars, regular expressions, and finite automata.

Project Link: https://starlight3dca.netlify.app/

GitHub Repo: https://github.com/sunyentan/starlight

Phase 1: Inspiration - Stars and Galaxy Formation

The motivation for this project stems from my passion for 3D spatial data and how we visualize it and my learnings from CSE 355, Theory of computing. When I combined these two ideas, I knew I also wanted to create a connection between stellar evolution and theory of computation.

In astrophysics, galaxies are formed as a result of gravity acting on cosmic matter where stars and planetary bodies emerge through accretion and self organization.

This project models from that behavior using computational growth principles where each “cell” represents a particle that follows growth rules like the gas clouds forming stars
Here are some key parallels between the cellular automata in this project and star formation:

Cellular Automata	**Star Formation**
Random Seeding	Proto-stars emerge in cosmic clouds
Nondeterministic Branching	Matter condenses in different locations
Interaction rules	Particles change over time
Bounded space	Simulation occurs within a defined universe

Phase 2: Computation Design (CFG, RegExp, DFA)

Before I started writing out a script for this project, I wanted to create a formal computational model of this process. The automaton was designed using the theory of computation concepts learned in class.

1. **Random cell movement represented by regular expression**

   Each cell undergoes movement defined by a set of random transitions in the 3d space, in six directions:
   a. U (Up): Move in the positive Y direction → (x,y+stepSize,z)
   b. D (Down): Move in the negative Y direction → (x,y−stepSize,z)
   c. L (Left): Move in the negative X direction → (x−stepSize,y,z)
   d. R (Right): Move in the positive X direction → (x+stepSize,y,z)
   e. F (Forward): Move in the positive Z direction → (x,y,z+stepSize)
   f. B (Backward): Move in the negative Z direction → (x,y,z−stepSize)

The sequence of moves in the 3d space can be represented like this:

$$(U \mid D \mid L \mid R \mid F \mid B)^{+}$$

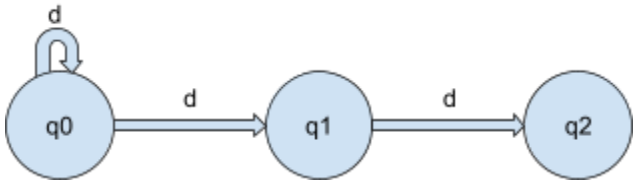
2. **Growth rule represented by context-free grammar**

   The growth rule can be expressed as a context free grammar which generates branching behavior:

   $Cell \rightarrow d\ Cell$ (Move in one direction)
   $Cell \rightarrow d\ Cell\ Cell$ (Spawn an extra branch)
   $Cell \rightarrow \varepsilon$ (Base case, the initial seed or when it reaches boundary)

3. **New cell generation decision making process represented by an NFA**

   The NFA below shows how each cell generates new states:



Phase 3: Technical system design and implementation

After defining the computational model using regular expressions, context-free grammars, and an NFA, the next step was to translate this into a functional 3D visualization system using Three.js, a javascript graphic library framework.

This phase involves defining the core architectural components, how the automaton functions in 3D space, and how users can interact with and modify the simulation.

Core Components:

1. Three.js Scene: 3D environment:
   The scene is initialized using this snippet of code in the .js file

```
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x222222); // color of background
```

2. Orbit controls:
   To allow users to view the automaton from any angle and zoom around freely, we need to add Orbit controls. Thus, we add the following:
   a. Perspective Camera: A camera that simulates realistic depth on a 2d screen
   b. Orbit controls: Allows users to rotate and zoom around the simulation

```
const camera = new THREE.PerspectiveCamera(
  75,
  window.innerWidth / window.innerHeight,
  0.1,
  1000
);
camera.position.set(25, 25, 25);

const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
```

3. Point-based Automaton:
   Particles are represented as THREE.Points. This approach is:
   a. Efficient as it uses GPU accelerated rendering
   b. Flexible as points can be colored, sized, and moved dynamically

   The following is the code for initializing the points:

```
let positions = [];
let colors = [];

let geometry = new THREE.BufferGeometry();
geometry.setAttribute('position', new
THREE.Float32BufferAttribute(positions, 3));
geometry.setAttribute('color', new THREE.Float32BufferAttribute(colors,
3));

const pointsMaterial = new THREE.PointsMaterial({
  size: 0.2,  // adjustable in the UI
  vertexColors: true,
});

const pointsCloud = new THREE.Points(geometry, pointsMaterial);
scene.add(pointsCloud);
```

4. Bounding Cube:
   Defines the containment field, which prevents the automaton from growing infinitely.

```
let boxSize = 20;
const boxMaterial = new THREE.LineBasicMaterial({ color: 0xffffff });
let boxGeometry = new THREE.BoxGeometry(boxSize, boxSize, boxSize);
let edges = new THREE.EdgesGeometry(boxGeometry);
const boxWireframe = new THREE.LineSegments(edges, boxMaterial);
scene.add(boxWireframe);
```

Additionally, users can adjust the cube size dynamically:

```
function updateCube(newSize) {
  boxSize = newSize;
  boxWireframe.geometry.dispose(); // remove old geometry
  let newBoxGeometry = new THREE.BoxGeometry(boxSize, boxSize, boxSize);
  let newEdges = new THREE.EdgesGeometry(newBoxGeometry);
  boxWireframe.geometry = newEdges;
}
```

5. User Controls:
   I created a control panel with parameters to adjust automaton behavior. This gives the user the flexibility to experiment with different situations.

   Since the code snippet for the control panel is long, I will include just 1 example. Please refer to the github repo for the full code.

```
<div>
     <label for="maxCellsRange">Max Cells:</label>
     <input type="range" id="maxCellsRange" min="1000" max="20000" step="100" value="10000">
     <input type="number" id="maxCellsNumber" min="1000" max="20000" step="100" value="10000">
</div>
```

   These elements are linked to JavaScript functions that update the automaton in real-time.

Key Features:

1. Step-based particle expansion: (CFG and NFA implementation)
   Each generation follows a stochastic rule, where new particles are spawned randomly in space while following constraints. It is controlled by:

```
function simulateGenerationStep() {
  let newActiveCells = [];
  for (let cell of activeCells) {
    let direction = randomUnitVector();
    let newPos = cell.position.clone().add(direction.multiplyScalar(stepSize));

    if (isInsideBox(newPos)) {
      let newCell = { position: newPos, generation: cell.generation + 1 };
      newActiveCells.push(newCell);
      allCells.push(newCell);
      positions.push(newCell.position.x, newCell.position.y, newCell.position.z);

      if (Math.random() < spawnExtraProbability) {
        let extraDir = randomUnitVector();
        let extraPos =
```

```
cell.position.clone().add(extraDir.multiplyScalar(stepSize));
        if (isInsideBox(extraPos)) {
          let extraCell = { position: extraPos, generation: cell.generation + 1 };
          newActiveCells.push(extraCell);
          allCells.push(extraCell);
          positions.push(extraCell.position.x, extraCell.position.y,
extraCell.position.z);
        }
      }
    }
  }
  activeCells = newActiveCells;
}
```

2. Speed & visualization controls
   Users can pause, step forward, step backward, and adjust simulation speed.

   Play/pause:

```
const pausePlayButton = document.getElementById("pausePlayButton");
pausePlayButton.addEventListener("click", function() {
  isPaused = !isPaused;
  pausePlayButton.textContent = isPaused ? "Play" : "Pause";
});
```

   Adjust speed:

```
const simSpeedRange = document.getElementById("simSpeedRange");
simSpeedRange.addEventListener("input", function(e) {
  simSpeed = parseFloat(e.target.value);
});
```

3. Color-based generation tracking
   To simulate the evolution of stars, each cell changes color as it ages. This makes younger stars appear blue and older stars appear red, mimicking real stellar evolution.

```
function updateColors() {
  const timeFactor = performance.now() * 0.0001;
  const newColors = [];
  for (let i = 0; i < allCells.length; i++) {
    const cell = allCells[i];
    let hue = (cell.generation * 0.05 + timeFactor) % 1;
    const col = new THREE.Color().setHSL(hue, 1, 0.5);
```

```
    newColors.push(col.r, col.g, col.b);
  }
  colors = newColors;
  geometry.setAttribute('color', new
THREE.Float32BufferAttribute(colors, 3));
  geometry.attributes.color.needsUpdate = true;
}
```

4. Nondeterministic movement (Regular expression implementation)
   The function `randomUnitVector()` is a key component in determining the movement of new cells in the simulation. It ensures that each new cell moves in a completely random direction in 3d space:

```
function randomUnitVector() {
  const theta = Math.random() * 2 * Math.PI;
  const phi = Math.acos(2 * Math.random() - 1);
  return new THREE.Vector3(
    Math.sin(phi) * Math.cos(theta),
    Math.sin(phi) * Math.sin(theta),
    Math.cos(phi)
  );
}
```

   How it works:
   a. Generating theta
      i. `theta = Math.random() * 2 * Math.PI` Represents a random rotation around the z-axis
      ii. The range is 0 to 2π which covers a full 360 degree rotation
   b. Generating phi
      i. `phi = Math.acos(2 * Math.random() - 1)` Determines how "high" or "low" the point is on the sphere
      ii. Using `acos(2 * Math.random() - 1)` ensures uniform sampling of points on a sphere
   c. Converting Spherical Coordinates to Cartesian (X, Y, Z)
      i. The unit vector is calculated using:
         1. x = sin(phi) * cos(theta)
         2. y = sin(phi) * sin(theta)
         3. z = cos(phi)

   This is done this way because if we just picked random x, y, z values between -1 and 1 and normalized them, the points would just cluster near the poles instead of being evenly distributed. Using polar and azimuthal angles ensure uniform distribution over the sphere. This creates “true” randomness in all directions.

5. History tracking for stepwise simulation (Next/previous gen button)

The function pushCurrentStateToHistory() is essential for enabling the pause/play and step-back/step-forward features in the simulation.

```
function pushCurrentStateToHistory() {
  generationHistory.push({
    activeCells: activeCells.map(cell => ({ position: cell.position.clone(),
generation: cell.generation })),
    allCells: allCells.map(cell => ({ position: cell.position.clone(), generation:
cell.generation })),
    positions: positions.slice(),
    colors: colors.slice()
  });
}
```

How it works:

Everytime a new generation is created, this function saves the current state of the simulation so that users can step back and forth between generations.

Design Choices and reasoning

Why Three.js and Not Other Graphics Libraries?

When choosing the right tools for this project, I went to research and experiment which one works best. I needed a graphic library to display the real time generation and rendering of the cells. Three.js was chosen based on accessibility, efficiency and the available resources it offered.

Below was a table of my comparison of graphic library frameworks:

Feature	Three.js	Babylon.js	WebGPU	Unity WebGL
Browser Compatibility	✅ Fully supported in modern browsers	✅ Fully supported	⚠️ Limited (Still in experimental stages)	❌ Requires Unity WebGL export
Performance (for 10000 particles)	✅ **Optimized for WebGL** (GPU accelerated)	✅ High performance, but slightly heavier API	⚠️ Future potential for higher GPU efficiency	❌ CPU-heavy, inefficient for many small objects
Ease of Use	✅ Simple API, widely documented	✅ Similar to Three.js but slightly more complex	❌ Lower-level API, requires manual shader programming	❌ Requires Unity interface and build process
Interactivity	✅ Full JS &	✅ Full JS &	❌ Requires	❌ Requires C#

(DOM & JavaScript)	HTML integration	HTML integration	WebAssembly or low-level APIs	scripts & compiled builds
Community & Learning Resources	✅ Large community, many tutorials	✅ Large community but slightly smaller than Three.js	❌ Still developing, limited documentation	❌ Harder to integrate into web-based applications

However, I do feel like WebGPU is the successor of Three.js, although it still needs time to develop and grow as a community.

Why a Web-Based Visualization Instead of Standalone Software?

I chose this project to be a web based visualization instead of a simple python script / standalone software due to the following reasons:

1. **Universal access**: Anyone with a browser and internet connection can access the tool without the need of installing anything
2. **WebGL leverage**: Using the client side GPU rendering for real time performance
3. **Extensibility**: Easy to integrate with HTML UI elements and increase interactivity and aesthetics

---

Why Points-Based Rendering Instead of Meshes?

To represent individual cells in the automaton, we had two main choices in Three.js:

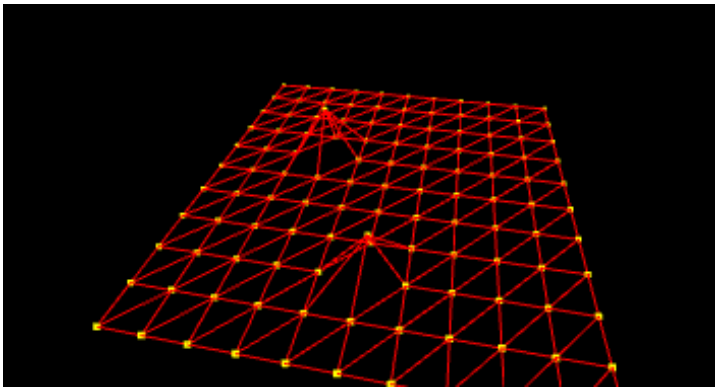
1. Use full 3D meshes (example: spheres for each cell)
2. Use point-based rendering (THREE.Points)

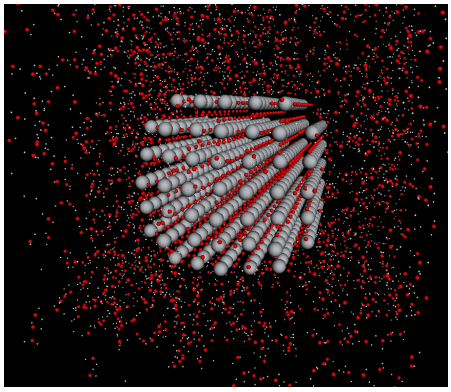
I decided to conduct an experiment to see which option was better performing and collected the following data of the max frame rate collected:

Machine Used: MacBook Air, 2024, M3, 13 inch, 16 GB, 8 cores

Rendering Method	10,000 cells	100,000 cells	1,000,000 cells
THREE.Mesh (spheres)	48 FPS	8 FPS	Crashed
THREE.Points	60 FPS	43 FPS	15 FPS

In the end, I chose THREE.points as they have faster frame rates. This could be because WebGL handles all the points in a single draw call, but Meshes require individual draw calls for each object which decreases performance.





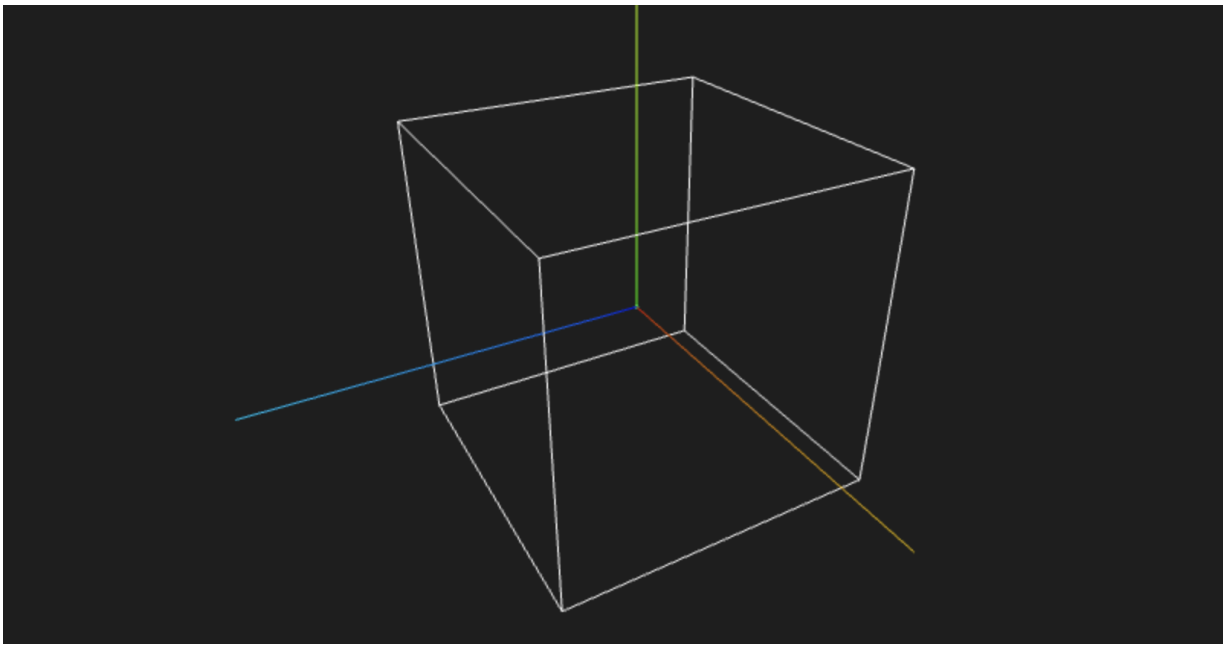
Why did I add a bounding cube?

The 3D cellular automaton grows within a 3D bounding cube instead of just an infinite space. This is because of the following reasons:

1. Without the bounds, the simulation could start expanding exponentially due to the CFG defined above, which could potentially crash the browser and computer
2. Galaxies are believed to be finite, but growing every second. Thus, I made the bounds adjustable, which keeps the simulation a realistic containment field

While I did test whether an infinite grid worked, these were my results of my experiment:

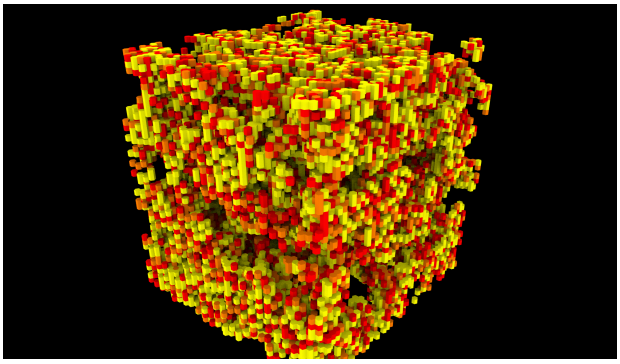
Method	Pros	Cons
Bounding Cube	Faster processing, easier to visualize	Less "natural" than an infinite universe
Infinite Grid	More freedom, no artificial boundaries	More demanding computations

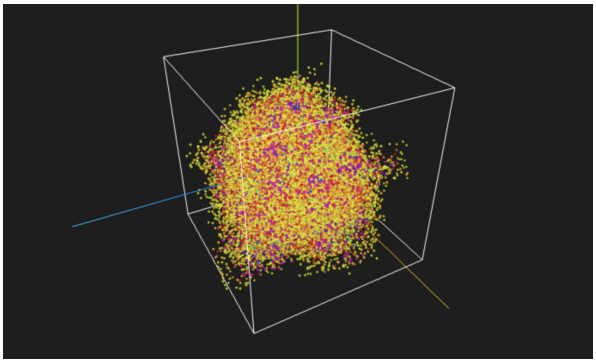


Why a stochastic growth model instead of a grid-based cellular automata?

Most traditional cellular automata, like Conway's game of life, are grid based. However, my automaton follows randomized movement rules, meaning that instead of having defined grid spots tightly next to each other, we have randomized direction movement rules the automaton follows. This is because:

1. It closely resembles cosmic structures. The stellar formation is random and not grid like
2. Each cell can freely expand in space, instead of having strict neighbor rules
3. Unpredictable behavior, which produces new pattern and cool structures



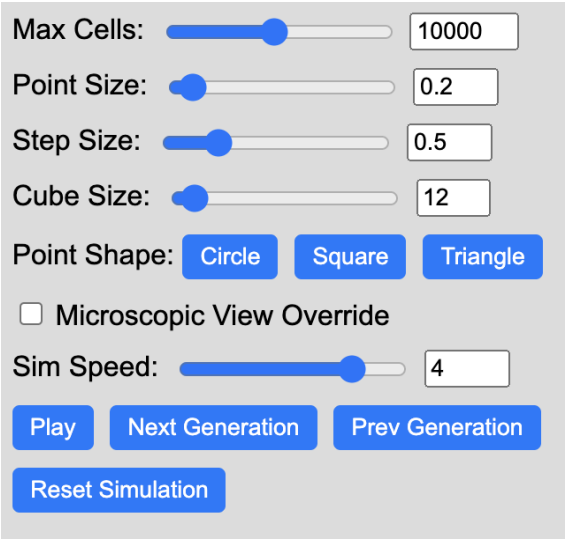


Evaluation of UI and interaction choices

To allow for experimentation and user interactivity, I implemented user controls for real time

adjustments:

Feature	Description
Max cells slider	Adjust the maximum number of cells generated in a session
Cell size slider	Adjust the size of the cell
Step size slider	Adjust the distance between each cell
Cube size slider	Modify the bounding cube size dynamically
Cell shape button	Change the shape of the cell from circle, triangle, and cube
Microscopic toggle	Enhance small scale details when zooming in
Speed slider	Adhyst the simulation pace
Pause/Play button	Allows users to control growth speed
Next/Previous Generation button	Enables stepwise simulation for analysis
Reset button	Deletes all cells



Reflection and key learnings

Working on this 3d cellular automaton project has been an insightful experience that merged computational theory with visual simulation. Initially, I was inspired by star formation and wanted to model how simple rules could lead to complex structures, similar to how galaxies emerge. Through this project, I deepened my understanding of how computational models can represent natural phenomena. It bridged my learnings from the classroom and allowed me to create a real project using the concepts like CFG and NFAs and how they can be useful

when designing programs.

One of the most significant challenges was bridging the gap between theoretical concepts (like CFG, NFAs, and regular expressions) and actual implementation in JavaScript using Three.js. While these formal definitions helped define the structure of the automaton, translating them into efficient, interactive code required additional considerations, such as performance optimizations, stochastic branching logic, and real-time rendering. The need for history tracking to allow stepwise control also introduced an interesting problem in maintaining state without losing previous generations, which I solved by implementing a generation history system with indexed snapshots.

Finally, working with Three.js expanded my understanding of 3d visualization and WebGL. It was rewarding to see mathematical models transform into interactive, aesthetic simulations. Adding glowing effects and user controls made the project more engaging and demonstrated how theory and implementation can come together to create something both functional and visually impressive. This project has reinforced my appreciation for the intersection of computing theory, graphics, and user interactivity, and I'm excited to apply these learnings in future work.

When you play around with the simulation, I realize that within the star formation, you can visualize "mini stars" where each cell is a star instead of all the cells forming a star.